AgentGS: Empowering Earth Science Agents with Plan Composition and Completion over Skill Hypergraphs

Anonymous Author(s)
Submission Id: TBD

Abstract

Complex geoscientific tasks often require multiple tools to work together and exchange intermediate results across computational steps. Existing methods commonly plan around specific tools, tying workflows to particular implementations, whereas reusing entire strategies makes it difficult to combine local steps from different strategies according to task requirements. We present AgentGS, a method for capability composition and tool orchestration over skill hypergraphs. AgentGS uses a three-layer Skill–Capability–Tool representation to decouple task strategies, reusable operations, and concrete tools, allowing it to organize an operational workflow before selecting its implementations. The skill hypergraph connects strategies through shared capabilities and provides a structured basis for composition across strategies. Given selected strategies, the system expands their required operations and prerequisite steps. When the current operation chain lacks necessary intermediate steps, it searches for feasible connecting branches along dependencies declared by other strategies. By jointly checking step dependencies, data compatibility, and tool availability, the system incorporates branches that satisfy the constraints without ambiguity, thereby reusing local steps from other strategies. It then selects concrete tools, propagates task data and intermediate results, and locally recovers failed steps. AgentGS can also organize newly composed capability workflows into candidate Skills for subsequent registration and reuse. On GeoToolEval, which contains 603 evaluation records across ten geoscientific domains, AgentGS achieves a mean Final Answer Score (FAS) of 0.8731. Configuration comparisons show higher scores for the full method on the reported overall and multistep process metrics. Some configurations provide only partial controls or did not actually activate the intended ablation, so their differences cannot be directly attributed to the corresponding modules.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; *Planning and scheduling*.

Keywords

LLM Agents, Compositional Planning, Tool Orchestration, Skill Hypergraph, Scientific Agent

ACM Reference Format:
Anonymous Author(s). 2027. AgentGS: Empowering Earth Science Agents with Plan Composition and Completion over Skill Hypergraphs. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 17 pages.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

1 Introduction

Earth science seeks to derive testable scientific conclusions from observational data, from climate change and critical minerals to energy resources and disaster risk. Its research frontiers are data intensive, computationally demanding, and interdisciplinary, with increasing emphasis on predictive systems science driven by big data and artificial intelligence [15]. Progress requires organizing distributed data, tools, and methods into reproducible, traceable analytical workflows.

Such workflows typically require data retrieval, spatial or temporal alignment, variable derivation, statistical modeling, and result verification. Researchers manually connect these stages, despite differences in data formats and reporting standards. Scientific agents built around large language models (LLMs) offer a way to automate this process by translating natural-language goals into multistep plans and coordinating tools [4, 5]. Crucially, later geoscientific operations depend on artifacts produced by earlier steps. Reliable tool orchestration is therefore essential to completing the analysis.

Existing geoscientific agents support multistep execution using frameworks such as ReAct [12]. GeoAgent [1] and Earth-Agent [5], for example, use planning–execution–reflection architectures. However, their execution units are typically individual calls, individual tools, or entire Skills. Tool-level planning couples workflows to concrete interfaces, whereas whole-Skill reuse makes it difficult to extract and recombine local operations from different strategies. An unresolved problem is how to compose operations across strategies while preserving their prerequisites and usage constraints, and subsequently select suitable tool implementations.

Evaluation presents a related challenge. EarthSE assesses geoscientific reasoning through questions and exploratory dialogue without requiring actual tool execution [11]. GeoBenchX [6], EarthBench [5], and GeoAgentBench [13] examine multistep tool use; the latter two also assess processes and outcomes. We evaluate tool composition and orchestration across geoscientific themes, distinguishing reference-answer agreement from completion of required operations within the same task. Failure analysis additionally requires parameters, artifacts, and invocation records; a matching answer or successful individual call does not establish a correct analysis.

We introduce AgentGS and GeoToolEval to address these challenges. Our contributions are:

- AgentGS separates Skills, Capabilities, and Tools and completes missing capabilities through a hypergraph. This supports operation reuse across strategies, constraint preservation, and executable tool selection.

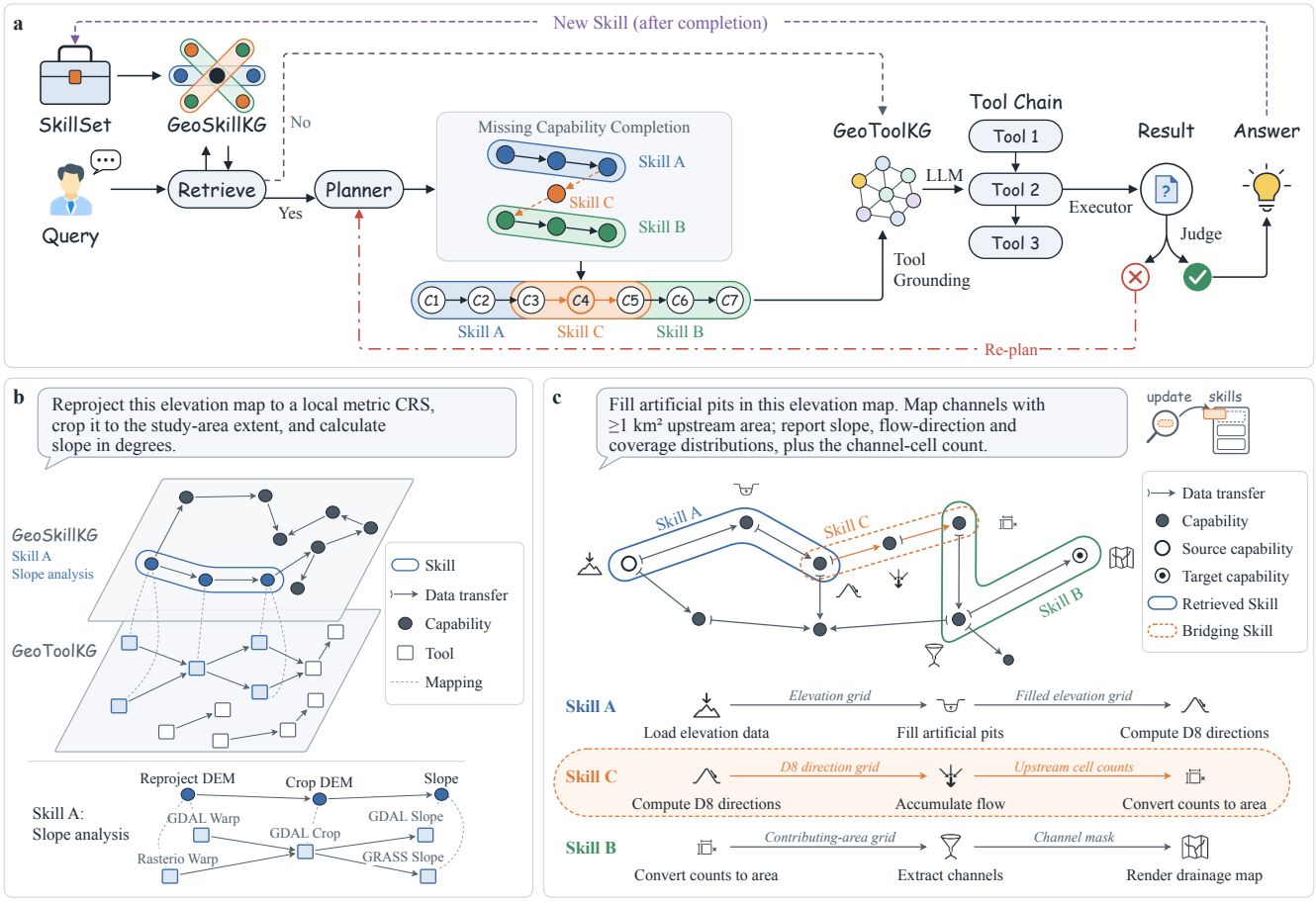


Figure 1: AgentGS hierarchical representation and overall workflow. (a) Retrieval, planning, completion, execution, verification, and Skill writeback. (b) Shared capabilities, strategy-specific dependencies, and tool implementations. (c) A connecting branch retrieved from another Skill. Verified compositions can be saved as candidate Skills.

- GeoToolEval comprises 603 evaluation records across ten geoscientific research-frontier themes. Single-step and multistep tasks support joint analysis of tool dependencies, composition, intermediate-artifact use, and final answers.
- AgentGS achieves a mean Final Answer Score (FAS) of 0.8731 on GeoToolEval, outperforming the evaluated models and agents. Multistep strata and configuration comparisons show answer and workflow differences, whose causes require execution traces and scrutiny of actual interventions.

2 Related Work

Planning and dependency representations. ReAct interleaves reasoning, actions, and observations [12]. ToolPlanner methods support task execution over large tool collections [8, 10], while LLM+P and Generalised BDI Planning introduce explicit planning and goal management [7, 9]. GeoAgent and Earth-Agent apply multistep planning and execution to geoscientific analysis [1, 5]. SciToolAgent orchestrates scientific tools through a knowledge graph [4]; HyperAgent constructs a tool hypergraph from input/output schemas and searches for supporting tools [14]. AgentGS instead represents shared operations as Capabilities and strategies as Skills. It preserves each operation's dependencies and usage constraints within its source strategy, allowing local cross-strategy reuse before tool selection.

Geoscientific evaluation. GeoBench and EarthSE evaluate knowledge understanding and scientific exploration without requiring actual tool calls [2, 11]. Scientific agents demonstrate execution of code and data-processing tasks [1, 4, 5]. More recent benchmarks explicitly examine orchestration: GeoBenchX evaluates multistage geospatial tasks against reference solutions [6]; Earth-Bench distinguishes autonomous planning from instruction following and records tool inputs and outputs [5]; GeoAgentBench checks invocation order, parameters, and map artifacts in a GIS sandbox [13]; and GeoNatureAgent Benchmark tests multitool analysis and error recovery with real APIs [3]. GeoToolEval extends evaluation across ten research themes and supports analysis of operation selection, dependencies, composition, artifact consumption, and final

answers within the same task. It also examines completion of necessary operations, helping distinguish missing-capability failures from execution errors.

3 AgentGS

AgentGS organizes scientific tool use around reusable operations and their dependencies. Given a scientific question and available resources, the agent retrieves relevant strategies, composes their local operations into a task-specific plan, completes missing connections, and grounds the result in concrete tool calls. Figure 1 summarizes this workflow. Its central design choice is to preserve the strategy-specific conditions under which an operation can be reused while allowing that operation to participate in a new composition.

3.1 Task Formulation and Design

Let q be a natural-language task, X its available input resources, and $\mathcal{T}$ the registered tool collection. A task can request both computed quantities and output artifacts: for example, a composite raster, a statistic calculated from that raster, and a comparison with a supplied reference. Solving the task requires determining the necessary operations, their data dependencies, admissible implementations, and concrete arguments.

A useful plan must meet three conditions. First, it must cover the requested operations and their prerequisites. Second, each operation must have an available implementation whose inputs can be supplied by X or by successful preceding operations. Third, the produced quantities and artifacts must support the requested answer. The first two conditions constrain workflow construction; the third requires checking the meaning of the actual results. A structurally valid plan alone does not establish scientific correctness.

AgentGS separates this problem into capability planning and tool execution. Capability planning determines what operations are needed and how they connect. Tool grounding chooses implementations under the resulting constraints, and execution resolves their arguments from observed data. This allows alternative implementations of an operation without discarding its role in the workflow. It also makes a missing intermediate operation distinguishable from a failed invocation of an already selected operation.

3.2 Skill–Capability–Tool Representation

The representation distinguishes strategies, shared operations, and executable interfaces (Figure 1(b)).

Skills as reusable strategies. A Skill describes a class of workflows through an objective, applicable inputs, expected artifacts, capability steps, and usage constraints. Each step specifies its operation, mandatory predecessors, required or optional status, and permitted implementations. Validation requirements and recovery policies describe how the strategy's results and failures should be handled. Reuse selects task-relevant local steps; an agent need not execute every operation in the original strategy.

Capabilities as shared operations. A Capability specifies an operation through its function, input/output types, preconditions, postconditions, and candidate tools. The same capability can occur in several Skills. Its occurrence retains the source strategy's constraints: two Skills may both use raster analysis but permit different tools or require different preceding transformations. Sharing a capability therefore provides a point of connection between strategies without making all its occurrences interchangeable.

Tools as implementations. A Tool is a callable interface with an identifier, parameter schema, input/output definitions, and runtime information. Capability–Tool mappings record which tools implement an operation and any operation-specific parameter constraints. The mapping is many-to-many: a capability can admit several implementations, and a tool can support several capabilities. Concrete files, values, and upstream artifacts are bound at execution time.

Skill hypergraph. Let S and C denote the registered Skills and Capabilities. Their membership defines

$$H = (C, E), \qquad E = \{e_s : s \in S\}, \qquad e_s \subseteq C. \tag{1}$$

Capabilities are nodes and each Skill is a hyperedge. Skills containing $\{a, b\}$ and $\{b, c\}$ intersect at capability b, providing a candidate connection for composition. Hyperedge membership records which operations belong to a strategy. It does not, by itself, specify the order or data flow between them.

Execution prerequisites are represented separately by a scoped relation $d = (s, U, c)$: within Skill s, capability c requires every predecessor in U. This is a conjunctive requirement. When an operation requires both a prepared raster and a reference value, supplying either input alone does not satisfy the declaration. Conditional order is used only when its stated condition applies. The strategy identity remains part of each relation, preventing a dependency from one Skill from being silently substituted for another's.

Capability instances and artifact bindings. A task-specific plan is

$$P = (V, D_d, D_c, \beta), \qquad v_i = (c_i, s_i, r_i), \tag{2}$$

where c_i is the operation, s_i its source Skill, and r_i its step/input role. D_d and D_c represent data and control dependencies, respectively; β binds inputs to task resources or outputs of preceding instances. A capability is a reusable definition, whereas an instance is one occurrence on particular inputs. Two operations on different rasters remain distinct even if they select the same tool. This distinction is essential for preserving multiplicity, provenance, and the identity of the artifact consumed downstream.

3.3 Plan Composition

Composition turns retrieved strategies into a task-specific set of operations. Skill retrieval uses the question, domain, and input types to supply candidate strategies. Selection then identifies applicable Skills and the operations needed for the requested outputs. The LLM makes these decisions within the retrieved declarations, retaining the relevant source Skills and their constraints.

Mandatory-predecessor expansion. Selected operations can depend on outputs not yet included in the plan. For a selected Skill s, let $R_s^{(0)}$ contain its initially required capabilities. Its dependency closure can be written as

$$R_s^{(k+1)} = R_s^{(k)} \cup \bigcup_{\substack{(s,U,c)\in\mathcal{D} \\ c\in R_s^{(k)}}} U, \tag{3}$$

where $\mathcal{D}$ is the set of declared mandatory dependencies. Expansion continues until no required predecessor is missing. This expression describes closure within a strategy; instantiation still preserves separate occurrences and input roles. Missing declarations and dependency cycles are diagnosed rather than accepted as executable plans.

Combining local fragments. Composition assembles the selected operations and their required predecessors into P. Composition retains the provenance and admissible tool scope of every occurrence, together with its data and control dependencies. Shared capability identity makes reuse discoverable, but does not justify merging operations on different inputs or changing a strategy's prerequisites.

Predecessor expansion only supplies dependencies already declared for the selected operations. It can leave a gap between fragments from different strategies. For instance, a compositing fragment may produce a raster while a comparison fragment expects a scalar statistic. Both fragments can be internally complete and still fail to form an executable analysis. Completion addresses these remaining gaps.

When no Skill is applicable, AgentGS retrieves tool candidates from GeoToolKG for direct tool-call planning. Both routes use the same execution and answer-generation interfaces. This fallback provides access to registered tools, but it does not supply the strategy knowledge absent from retrieval.

3.4 Plan Completion over the Skill Hypergraph

Completion examines whether the composed plan contains the required operations, whether their inputs are supplied, and whether admissible implementations are available. These are different failure modes: an omitted statistic requires another operation, an unresolved artifact reference requires a valid producer, and a missing implementation cannot be repaired merely by ordering the existing steps.

Supporting-capability completion. The system searches registered knowledge for operations supporting an identified gap. Activated and directly compatible Skills can supply missing task operations, while explicit compatibility relations guide the search for unsatisfied dependencies. Insertions are followed by prerequisite expansion and conflict checks. Retrieval cues help identify candidates, but keyword overlap alone is not evidence that an operation is scientifically necessary.

Directed bridging. Bridging connects active fragments using a directed branch from an inactive Skill. Let u be an output-boundary capability of active Skill A and v an input-boundary capability of active Skill B: u has no successor in A's active local dependencies, and v has no predecessor in B's active local dependencies. An inactive provider Skill can declare

$$\pi = (u, m_1, \ldots, m_k, v), \qquad k \geq 1, \tag{4}$$

where at least one intermediate capability is inactive. The provider supplies both the intermediate operations and their declared connection to the endpoints. Figure 1(c) illustrates this reuse of a local branch.

Search fixes the active capability set at its start. Hypergraph membership identifies inactive Skills containing the two endpoints; explicit dependencies and applicable conditional order determine directed simple paths within each provider. Co-occurrence and list position do not establish direction. Candidates for an endpoint pair are skipped when an indirect data path already connects that pair. Newly inserted operations do not recursively extend this search.

Branch validity. A candidate branch must satisfy five conditions. Its direction must be declared; all mandatory predecessors must belong to the active set or the candidate branch; adjacent operations must share compatible concrete input/output types; every required operation must have an available tool within its permitted scope; and both individual and joint insertions must preserve acyclicity. Generic types alone do not establish compatibility. External active predecessors retain their original constraints rather than inheriting the provider's scope.

Acceptance and recomposition. For an ordered endpoint pair, only a unique valid branch is inserted automatically. Multiple valid candidates produce an ambiguity report. Insertion preserves the entry instance's dependencies and implementation constraints, adds the intermediate operations, and updates the exit's dependencies, tool scope, and bindings according to the accepted provider. Recomposition then checks for unmet prerequisites or additional operations outside the accepted branch. A candidate that fails these checks is rejected.

This procedure makes bridge insertion explicit and inspectable. It establishes compatibility with declared workflow knowledge, rather than a guarantee that every inserted operation is required by the task's scientific meaning. Appendix A provides the detailed algorithm and acceptance conditions; Appendix E works through a complete example.

3.5 Grounding, Execution, and Verification

A completed capability plan describes operations and dependencies. Grounding converts it into execution instances with concrete tool identifiers, arguments, and references to upstream results.

Constrained tool selection. For each capability instance, the candidate set is restricted by its source Skill, operation requirements, tool availability, and interface constraints. GeoToolKG supplies tool descriptions and associations for retrieval; those associations do not override the plan's dependencies. The LLM selects implementations within the admitted scope. Required operations and scoped ordering are checked again after the concrete chain is produced.

Explicit input propagation. Each execution instance has an identity and declares its predecessors. Input bindings name a producing instance and a field in that instance's observed output. At execution time, the system resolves these references and validates the resulting arguments against the selected tool's schema. A consumer cannot substitute an unrelated earlier call merely because it used the same tool name. Unknown predecessors, unresolved references, and incompatible arguments prevent invocation.

This separation is especially important for scientific artifacts. In a compositing task, the statistics call must consume the generated composite, not another raster mentioned in the question. Likewise, a comparison must use the statistic actually returned by its producer.

The execution record links these inputs to their sources, making artifact use inspectable independently of the final answer.

Failure handling. Execution failures produce structured feedback for bounded argument repair or replanning when recovery is enabled. Literal arguments may be corrected, while observed upstream bindings remain authoritative. Replanning validates instance identities, dependencies, and scopes again. Reuse of a successful result requires unchanged inputs and instance semantics; it is not inferred from a common tool-name prefix. The runtime conservatively reruns file-backed and external-data computations across plan attempts, while allowing reuse of explicitly supported pure scalar computations when their complete dependencies are verified.

Tool recovery, initial plan repair, and transport retries have separate roles and budgets. Disabling execution recovery terminates the scientific chain after an execution failure while retaining initial planning and input validation. Engineering details and the distinction between these budgets appear in Appendix B.

Verification and answer generation. The answer is constructed from observed results and artifacts. Verification checks whether the requested outputs are supported and records unfinished operations when execution is incomplete. Successful invocation, evidence consistency, and agreement with a benchmark reference are distinct properties; the benchmark evaluates the last of these independently.

A verified composition can also be represented as a candidate reusable Skill, preserving its input roles, dependencies, and usage constraints. This is a strategy representation over existing operations, not an expansion of the underlying tool collection. The present results do not establish a performance or cost benefit from autonomous Skill accumulation; the candidate-registration design is described in Appendix A.3.

4 GeoToolEval

GeoToolEval is a dataset for evaluating scientific answers and the tool workflows used to obtain them. It covers ten geoscientific research themes and pairs each task with input resources, a reference answer, and a reference invocation chain. This supports separate analysis of outcome quality, tool coverage, and invocation order within the same evaluation record.

4.1 Evaluation Objectives and Task Design

Scientific tool use involves more than selecting a function with a relevant description. An agent must identify the intended data, instantiate the analysis with appropriate parameters, and propagate results to later operations. GeoToolEval includes tasks that exercise these requirements at different orchestration lengths.

Single-step tasks assess resource identification, parameter determination, and invocation under a registered interface. Multistep tasks additionally require composing operations and using intermediate results. For example, if a question requests the mean of a composite raster, independently computing means of the input rasters does not establish that the requested artifact was produced and analyzed. The answer and the tool trace expose different aspects of this distinction.

Tasks span three-dimensional Earth structure, geoscientific big-data analysis, critical minerals, climate change, energy resources, the global ocean carbon cycle, the deep-time biosphere, carbon capture and storage, remote sensing, and natural-disaster event statistics. This breadth covers different data objects and analytical operations. The themes are not equally sized: their counts reflect resource availability and screening outcomes.

4.2 Task Sources and Construction

The construction combines two sources of tasks.

Adapted question-answer tasks. Questions selected from existing resources retain their source questions and reference answers, while the required operations are mapped to the registered tool interfaces. Geoscientific big-data analysis includes 40 selected GeoQuestions1089 tasks. Remote sensing includes 49 of Earth-Bench's 248 tasks [5], retaining questions, choices, and answers. Critical-mineral and climate-change tasks are selected from habitable_earth_dataset by theme.

Tasks derived from scientific data. These tasks specify identifiable objects, variables, and computational relationships. Reference results are obtained through corresponding processing and calculations. Sources include Earth-structure models and earthquake catalogs, mineral and energy inventories, ocean-carbon observations, paleobiological data, synthetic carbon-storage monitoring data, and disaster-event records. Table 1 identifies the sources by theme; Appendix C gives further construction details.

Reference workflows and record format. Each evaluation record contains a natural-language question, resource locators, a reference answer, an answer type, and reference calls. A call identifies the registered tool, its parameters, and relevant input/output information. The reference chain expresses one implementation of the requested analysis under the exposed interfaces. Existing tasks retain source identifiers or locators, and data-derived tasks retain derivation records and computational scripts.

Interface adaptation determines the reference-chain length. A tool wrapper can encapsulate several processing operations in one invocation; therefore, source-dataset labels such as single-tool or multitool cannot be transferred without examining the registered interfaces.

Quality control. Construction checks source traceability, required fields, tool-name and parameter consistency, and sample execution. For data-derived tasks, retained calculations support recalculation of reference quantities. For adapted tasks, source answers provide an independent record of the target output. These checks have different scopes: a parseable invocation is not proof of scientific correctness, and successful sample execution does not certify all records. Construction validation is kept separate from the complete benchmark evaluation.

4.3 Dataset Composition

GeoToolEval contains 603 evaluation records: 424 have one reference invocation, 119 have two or three, and 60 have at least four. Its 179 multistep records constitute 29.68% of the dataset.

Table 1: GeoToolEval task sources and reference tool-invocation chain lengths. Counts refer to evaluation records.

Task theme	Main data or question sources	1 step	2–3	≥ 4	Total
Three-dimensional Earth structure	LITHO1.0, SEMUCB-WM1, USGS earthquake catalog	85	15	0	100
Geoscientific big-data analysis	Selected GeoQuestions1089 subset	40	0	0	40
Critical minerals	Habitable-Earth questions; MRDS and cross-theme climate data	44	14	0	58
Climate change	Habitable-Earth questions; paleoclimate, GISTEMP, SSP, and cross-theme mineral data	31	16	1	48
Energy resources	MRDS, GEM, OPSD, World Bank, and population data	100	0	0	100
Global ocean carbon cycle	SOCAT, GLODAP, SNAPO-CO_2	100	0	0	100
Deep-time biosphere	BioDeepTime	12	7	5	24
Carbon capture and storage	Synthetic SCCS-OFPOSDU CCS–MRV data	12	5	7	24
Remote sensing	Selected Earth-Bench subset	0	7	42	49
Natural-disaster event statistics	EM-DAT International Disaster Database	0	55	5	60
Total		**424**	**119**	**60**	**603**

The critical-mineral and climate-change themes share nine cross-theme questions included in both themes. Thus, 603 denotes evaluation records rather than 603 distinct question texts. The main score weights records equally, so these repeated questions contribute through both theme assignments.

A step denotes a reference tool invocation, not an elementary scientific operation or a direct measure of scientific difficulty. Energy-resource and ocean-carbon tools, for example, can perform multiple internal operations within one call. Chain-length strata characterize orchestration demands under the current interfaces. They should be interpreted jointly with domain and wrapper design.

Long chains are unevenly distributed across themes. Remote sensing accounts for 42 of the 60 records with at least four reference calls. Consequently, changes across chain-length groups can also reflect domain-specific inputs and tools. We preserve both domain and chain-length views rather than treating either as a controlled difficulty scale.

4.4 Evaluation Metrics

We score each record separately and average over the declared evaluation population, retaining failures. The three metrics below distinguish final-answer agreement, successful-tool coverage, and relative invocation order.

Final Answer Score (FAS).. FAS lies in $[0, 1]$ and preserves partial credit. Scoring depends on answer type: choice normalization for multiple-choice answers, matching rules for dates and identifiers, numerical tolerances for scalar quantities, item matching for multiple values or text items, and field-level comparison for structured answers. Empty answers receive zero. For N records,

$$\overline{FAS} = \frac{1}{N}\sum_{i=1}^{N} FAS_i. \tag{5}$$

The main experiment uses $N = 603$. Domain and chain-length groups average their own records. This record-weighted mean is distinct from a domain-balanced macro-average or the rate of fully correct answers.

Tool Selection F1 (TSF1). Let G_i be the reference sequence and A_i the actual successful-call sequence for record i. Calls are matched one to one by tool name, treating the two sequences as multisets and retaining multiplicity. If m_i calls match,

$$\mathrm{TSF1}_i = \frac{2m_i}{|G_i| + |A_i|}. \tag{6}$$

This is the harmonic mean of successful-call precision and reference-call recall. Missing tools, irrelevant successful calls, and repeated successful invocations affect the score. Two required calls to the same tool remain two occurrences.

Tool Order Score (TOS).. The longest common subsequence measures preservation of relative order:

$$\mathrm{TOS}_i = \frac{\mathrm{LCS}(G_i, A_i)}{|G_i|}. \tag{7}$$

For reference order read–filter–count and actual order read–count–filter, TSF1 is 1 while TOS is 2/3. Extra calls do not necessarily lower TOS, which is why it is reported with TSF1.

Both process metrics are zero when no invocation succeeds. Failed invocations do not enter the successful-call sequence, but failed tasks remain in the average. All three metrics are reported separately; we do not combine them into a single measure of scientific correctness.

4.5 Interpreting Answer and Process Scores

A reference chain records one solution under the available interfaces. An alternative valid implementation may have lower name-based process scores, while a chain with the expected names and order can still consume the wrong artifacts or use incorrect parameters. TSF1 and TOS therefore measure agreement with reference workflows rather than semantic equivalence of arbitrary analyses.

The metrics support complementary questions. FAS measures whether the returned answer matches the reference. TSF1 identifies omissions and excess successful calls relative to the reference chain. TOS measures whether matched tools occur in a compatible relative order. Explaining why an answer is wrong, or whether a different workflow is valid, requires inputs, outputs, and dependency traces in addition to these aggregates.

5 Results

We evaluate AgentGS along three dimensions: answer quality on GeoToolEval, agreement with reference tool workflows, and performance in the external GeoBenchX environment. Configuration-level analysis of the method's components is presented separately in Section 6.

5.1 Experimental Setup

Evaluation population. The main comparison includes all 603 GeoToolEval records, using the metrics in Section 4.4. Every record contributes to the relevant average, including execution failures and incomplete answers. Results are also grouped by domain and reference-chain length. The 57-record configuration study is a separate experiment and is not used as the main population.

Backbone and baselines. AgentGS uses GPT-5.6 Terra. GeoAgent [1] and EarthAgent [5] use the same backbone and are evaluated in the same registered tool environment. Direct-answer baselines are DeepSeek V4 Pro, GPT-5.6 Terra, GPT-5.4, Qwen-Plus, and GeoGPT. They are compared by FAS; tool-trace metrics are inapplicable to their direct answers.

The shared-backbone agent comparison examines complete orchestration systems. The direct-answer comparison additionally changes access to tool execution and external computation. Neither comparison alone isolates a particular AgentGS module. The configuration study addresses narrower changes, subject to the intervention limitations described in Section 6.

Result provenance. AgentGS's reported main FAS is retained from its first complete evaluation. Its process metrics are calculated from the recorded 603-record end-to-end experiment. These aggregates retain their respective provenance and are not assumed to be a paired per-record analysis of the same executions. Appendix D summarizes the available result sources and the remaining reproducibility information. No repeat-run variance or significance estimates are supplied by the reported experiments.

5.2 Answer Quality on GeoToolEval

Table 2 reports the baseline comparison. AgentGS achieves mean FAS of 0.8731, compared with 0.7804 for GeoAgent and 0.0862 for EarthAgent. Its absolute advantage over GeoAgent is 9.27 percentage points.

Among the direct-answer models, DeepSeek V4 Pro obtains the highest reported FAS, 0.5162. Direct answers from AgentGS's GPT-5.6 Terra backbone score 0.3137. The corresponding AgentGS differences are 35.69 and 55.94 percentage points. These comparisons are consistent with the value of executing task-specific analysis in the reported tool environment, although the system-level gains combine resource access, planning, execution, and answer generation.

Figure 2 presents the overall and domain-specific answer scores. The overall mean weights individual records, so larger themes contribute more to the aggregate. The domain view complements the aggregate by showing variation across the ten task backgrounds.

Table 2: GeoToolEval baseline comparison. AgentGS process metrics are per-record averages from the present 603-record end-to-end experiment; FAS values are retained from the source manuscript. Direct-answer language models have no tool traces; their process metrics are not applicable (—).

Method	FAS	TSF1	TOS
AgentGS (GPT-5.6 Terra)	**0.8731**	0.9170	0.9166
GeoAgent (GPT-5.6 Terra)	0.7804	0.8663	0.7911
EarthAgent (GPT-5.6 Terra)	0.0862	0.1182	0.1390
DeepSeek V4 Pro	0.5162	—	—
GPT-5.6 Terra	0.3137	—	—
GPT-5.4	0.3069	—	—
Qwen-Plus	0.2918	—	—
GeoGPT	0.2500	—	—

Table 3: AgentGS by reference-chain length on GeoToolEval. Process scores for the single-step stratum are not supplied in the reported breakdown (—).

Reference calls	Records	FAS	TSF1	TOS
1	424	0.9425	—	—
2–3	119	0.7557	0.9779	0.9832
≥ 4	60	0.6167	0.8013	0.7785

5.3 Tool Coverage and Invocation Order

AgentGS obtains mean TSF1 of 0.9170 and TOS of 0.9166. GeoAgent obtains 0.8663 and 0.7911, while EarthAgent obtains 0.1182 and 0.1390. AgentGS's absolute differences from GeoAgent are 5.07 percentage points in coverage and 12.55 points in relative order.

The two metrics characterize different aspects of a workflow. GeoAgent scores higher on tool selection than on order. AgentGS has similar aggregate scores for these two metrics. Because TSF1 and TOS have different denominators and sensitivities, this does not yield an exact count of ordering failures. It indicates which aspect of reference-workflow agreement should be examined in the underlying traces.

AgentGS's higher answer and process scores provide complementary aggregate evidence. They do not show that every additional matched invocation improved an answer. Such an explanation requires aligning the answer and execution records, inspecting parameters, and verifying that later calls consumed the intended intermediate artifacts.

5.4 Performance across Workflow Lengths

Table 3 separates the reported AgentGS scores by reference-chain length. FAS decreases from 0.9425 on single-step records to 0.7557 on two- to three-step records and 0.6167 on records with at least four calls.

On the 119 two- to three-step records, TSF1/TOS are 0.9779/0.9832. On the 60 longer records, they are 0.8013/0.7785. Long workflows remain challenging for answer generation and tool orchestration. In the shorter multistep group, high process agreement

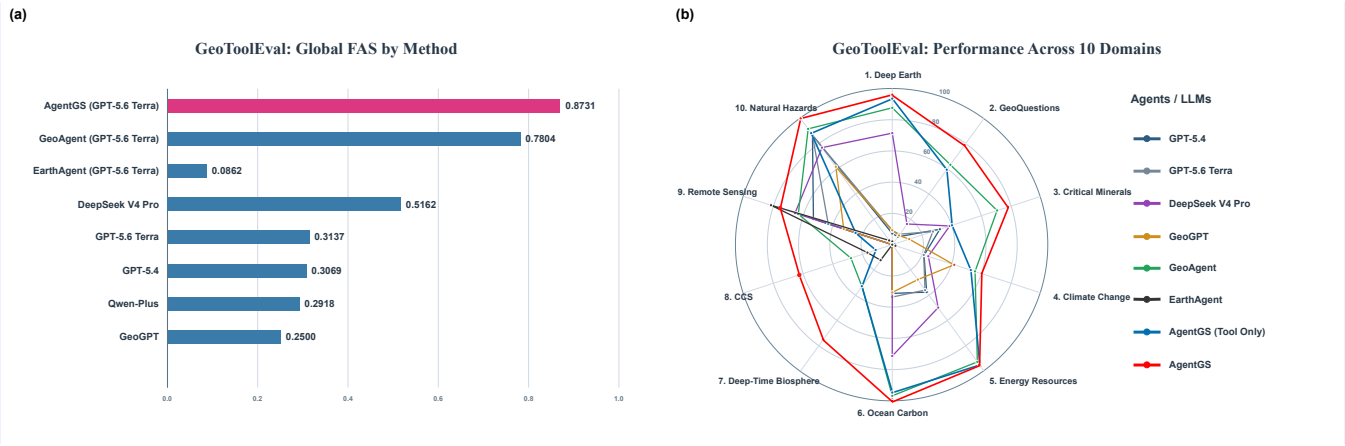


Figure 2: Overall and domain-specific performance of AgentGS and baselines on GeoToolEval: (a) mean FAS across all evaluation records; (b) mean FAS across the ten geoscientific domains.

Table 4: Evaluation results on GeoBenchX. Both methods use GPT-5.6 Terra.

Method	Mean score (0–2) ↑	Full-match rate (%) ↑
ReAct	1.1980	53.47
AgentGS	**1.3762**	**62.87**

coexists with lower answer scores. Call names and order therefore cannot replace answer evaluation.

The strata differ in domain composition as well as length. In particular, 70% of the longest-chain group belongs to remote sensing. The observed decline does not isolate the effect of adding one operation to an otherwise fixed task. It characterizes the current benchmark groups, motivating controlled comparisons that keep scientific inputs and task families comparable.

5.5 External Evaluation on GeoBenchX

The GeoBenchX evaluation [6] changes the registered tool environment. We integrate its official tools and construct corresponding Skills, Capabilities, scoped dependencies, and implementation mappings. This evaluates a configured deployment of the framework in another environment; dedicated knowledge construction is part of that deployment.

AgentGS and the official ReAct baseline are each reported on 202 tasks using GPT-5.6 Terra. The same model serves as the single judge under reference-solution matching: scores of 0, 1, and 2 denote mismatch, partial match, and match. Mean score and full-match rate are separate from GeoToolEval FAS.

AgentGS obtains a mean score of 1.3762 versus ReAct's 1.1980, an absolute difference of 0.1782 on the 0–2 scale. Full-match rates are 62.87% and 53.47%, a difference of 9.40 percentage points. Both reported aggregates favor AgentGS.

Exact paired task membership and matched execution conditions remain to be verified for these results. The single model judge also provides no estimate of agreement across evaluators. We therefore treat the comparison as reported external performance, without claiming controlled zero-shot transfer or attributing the difference solely to the planning representation.

6 Ablation and Analysis

We examine representation, dependency expansion, completion, tool selection, and recovery. The study reports answer scores alongside tool coverage and order. We distinguish the intended intervention from the intervention actually established by the reported run.

6.1 Study Design and Intervention Scope

The fixed subset covers all ten domains and contains 57 records: 22 single-step, 20 two- to three-step, and 15 with at least four reference calls. Its 35 multistep records account for 61.40%, compared with 29.68% in the complete dataset. The subset therefore emphasizes orchestration more heavily and should not be treated as a replacement for the 603-record evaluation.

Full is the reference configuration within this study. Its overall and multistep FAS are 0.6387 and 0.5333, respectively. These values belong to the subset experiment; neither its composition nor its run provenance supports directly equating them with the main evaluation.

A1: representation. The control asks the model to plan directly in tool instances while retaining strategies, dependencies, and candidate tools. It targets the organization of planning decisions rather than removing all strategy knowledge. The reported run implements only a partial control. Its score difference therefore cannot isolate the value of hierarchical representation.

A2: mandatory-predecessor expansion. The intended intervention retains selected requirements and dependency declarations but disables automatic predecessor closure. Completion remains available and may restore omitted prerequisites. The supplied result record states that the intended A2 intervention was not activated.

Its numbers are retained for provenance but are not evidence for an expansion effect.

A3 and A4: completion. A3 keeps gap diagnosis but disables supporting-capability insertion, including bridging. A4 disables only directed bridging while retaining the other completion mechanisms. Their scopes distinguish general support insertion from the additional connection supplied by an inactive provider Skill.

A5: implementation selection. The intended control chooses the first eligible tool in a fixed order while preserving operation requirements, candidate scope, and argument validation. The reported run changes only part of ranking, so it does not establish the effect of removing all selection optimization.

A6: execution recovery. A6 disables recovery after a scientific execution failure. Initial planning, grounding, and binding remain enabled. The relevant contrast is whether failure feedback can repair an execution or trigger replanning, rather than whether initial inputs are validated.

The present runtime exposes explicit controls for these interventions, but control availability does not retroactively establish that a reported experiment used them. Appendix D distinguishes current intervention semantics from the provenance of these scores.

6.2 Planning Representation and Dependency Expansion

Relative to Full, A1 lowers overall FAS by 16.39 percentage points and multistep FAS by 12.38 points. Its multistep TSF1/TOS are 0.8147/0.8295, compared with Full's 0.8862/0.8952. Thus, the reported configuration differs in both answers and successful-call sequences. Because A1 is only a partial control, these changes cannot be assigned solely to planning at the capability level.

A2 has overall and multistep FAS decreases of 16.27 and 12.38 points. Its multistep TSF1/TOS are 0.8302/0.8588. The inactive intervention prevents interpreting these differences as the effect of predecessor expansion, regardless of their magnitude.

There is also a structural interaction between expansion and completion. If completion inserts a prerequisite omitted by the expansion control, the final plan may remain unchanged even when the control is active. Conversely, a model can explicitly include necessary operations without automatic closure. Isolating expansion requires inspecting initial requirements, predecessor additions, support additions, and unresolved gaps, rather than inferring the intervention from the final score alone.

6.3 Supporting-Capability Completion and Bridging

A3 obtains overall FAS of 0.4624 and multistep FAS of 0.3802, decreases of 17.63 and 15.31 points from Full. Both scopes show lower answer agreement. The multistep decrease is smaller than the overall decrease, so this result does not support a claim that completion has a larger effect on multistep tasks in this subset.

A4 obtains overall and multistep FAS of 0.6160 and 0.4913, decreases of 2.27 and 4.20 points. It retains more of the Full configuration's answer performance than A3. The two configurations remove different sets of mechanisms, however, so their difference is not an independently measured additive contribution of bridging.

Process scores further separate their behavior. On multistep records, A3 obtains TSF1/TOS of 0.8364/0.8438, whereas A4 obtains 0.8249/0.8652. A4 has lower coverage agreement but higher order agreement than A3. This divergence shows why a single process score would obscure changes in successful-call sequences.

A bridge-specific explanation requires evidence that the task exposes a disconnected pair of active fragments, an admissible provider branch was available, and the inserted operation consumed and produced the expected artifacts. The case in Appendix E explains these conditions for a composite-raster workflow. It is a mechanism illustration rather than a measured frequency of bridge use across the dataset.

6.4 Tool Selection and Execution Recovery

A5 increases overall FAS from 0.6387 to 0.6415 but decreases multistep FAS from 0.5333 to 0.5184. Its multistep TSF1/TOS are 0.8133/0.8414, decreases of 7.29/5.38 percentage points. Answer and process changes therefore do not have a uniform direction across the reported scopes. In view of the partial ranking intervention, these results describe the available configuration rather than establish a general selection benefit.

A6 obtains overall and multistep FAS of 0.5972 and 0.4613, decreases of 4.15 and 7.20 points. Its multistep TSF1/TOS are 0.7652/0.7985, decreases of 12.10/9.67 points. The coverage decline is the largest among the six reported multistep configurations. This is consistent with recovery affecting how much of a successful-call sequence is completed, but the aggregate does not identify which failed calls were recoverable.

Trace-level attribution should distinguish argument corrections, alternative implementations, replanned operations, and exhaustion of execution budgets. A resumed scalar computation and a rerun of a file-producing step also have different costs. The reported scores contain no matched latency or token-cost analysis, so they do not establish the efficiency of recovery.

6.5 Joint Interpretation of Answer and Workflow Evidence

Full has the highest reported overall and multistep process scores: 0.9249/0.9392 and 0.8862/0.8952, respectively. Its process scores come from the supplementary 57-record experiment, while A1–A6 use records from the formal worksheet. The intervention and provenance limitations remain relevant even when all process differences favor Full.

The combined results identify three questions for mechanistic analysis. Which declared operations are missing before execution? Which consumers receive the intended upstream artifacts? Which failures can be repaired without changing the scientific objective? These questions connect the model's decisions to scientific outputs more directly than tool-name agreement alone. The current aggregates establish configuration differences; a causal account of individual components additionally requires matched runs and aligned execution traces.

Table 5: Configuration comparisons on the fixed 57-record subset. A1 is a partial control, A2 did not activate the intended ablation, and A5 changes only part of tool ranking. ΔFAS is the full method's mean FAS minus the corresponding configuration's mean FAS, in percentage points; positive values indicate lower performance than the full method.

Configuration	Mean FAS	Multistep FAS	ΔFAS (pp)
Full method	0.6387	0.5333	—
A1: Direct tool-level planning	0.4748	0.4095	16.39
A2: No predecessor expansion	0.4760	0.4095	16.27
A3: No independent completion	0.4624	0.3802	17.63
A4: No directed bridging	0.6160	0.4913	2.27
A5: No tool-selection optimization	0.6415	0.5184	−0.28
A6: No execution recovery	0.5972	0.4613	4.15

Table 6: Ablation process metrics. Multistep scores cover the 35 subset records with at least two reference calls. Each column averages per-task scores over its corresponding records. Full uses the supplementary 57-task experiment; A1–A6 use per-task records from the formal 57-task worksheet.

Configuration	Mean TSF1	Mean TOS	Multistep TSF1	Multistep TOS
Full method	0.9249	0.9392	0.8862	0.8952
A1: Direct tool-level planning	0.8599	0.8778	0.8147	0.8295
A2: No predecessor expansion	0.8694	0.8958	0.8302	0.8588
A3: No independent completion	0.8732	0.8865	0.8364	0.8438
A4: No directed bridging	0.8603	0.8997	0.8249	0.8652
A5: No tool-selection optimization	0.8503	0.8851	0.8133	0.8414
A6: No execution recovery	0.8208	0.8588	0.7652	0.7985

7 Conclusion

We present AgentGS and GeoToolEval for tool orchestration in Earth science. AgentGS separates reusable strategies, shared operations, and concrete implementations through a Skill–Capability–Tool representation. It composes local workflow fragments while retaining their dependencies and usage constraints, then searches the skill hypergraph for admissible operations that complete missing connections. Explicit grounding and artifact bindings connect this plan to executable scientific analysis.

GeoToolEval supplies 603 evaluation records across ten research themes, with reference answers and tool workflows that support separate measures of answer agreement, tool coverage, and invocation order. AgentGS achieves reported mean FAS of 0.8731, compared with 0.7804 for the strongest reported agent baseline, and also obtains higher aggregate process scores. The configuration study exposes answer and workflow differences while identifying limits on attributing them to individual mechanisms.

The resulting framework makes composition and completion inspectable at the level of operations, dependencies, and observed artifacts. Further evaluation should examine unseen capability combinations, controlled changes in the tool environment, and the utility of verified compositions as reusable strategies. Broader scientific reliability also requires validating task semantics and resource use beyond agreement with a reference call sequence.

References

[1] Yuxing Chen, Weijie Wang, Sylvain Lobry, and Camille Kurtz. 2024. An LLM Agent for Automatic Geospatial Data Analysis. arXiv:2410.18792 doi:10.48550/arXiv.2410.18792

[2] Cheng Deng, Tianhang Zhang, Zhongmou He, Yi Xu, Qiyuan Chen, Yuanyuan Shi, Luoyi Fu, Weinan Zhang, Xinbing Wang, Chenghu Zhou, Zhouhan Lin, and Junxian He. 2023. K2: A Foundation Language Model for Geoscience Knowledge Understanding and Utilization. arXiv:2306.05064 doi:10.48550/arXiv.2306.05064

[3] Gabriel Diaz-Ireland, Diego Prieto-Herráez, Mario García Peces, Javier Velázquez, and Devika Jain. 2026. GeoNatureAgent Benchmark: Benchmarking LLM Agents for Environmental Geospatial Analysis Across Frontier and Open-Weight Foundation Models. arXiv:2606.12821 doi:10.48550/arXiv.2606.12821

[4] Keyan Ding, Jing Yu, Junjie Huang, Yuchen Yang, Qiang Zhang, and Huajun Chen. 2025. SciToolAgent: A Knowledge Graph-Driven Scientific Agent for Multi-Tool Integration. arXiv:2507.20280 doi:10.48550/arXiv.2507.20280

[5] Peilin Feng, Zhutao Lv, Junyan Ye, Xiaolei Wang, Xinjie Huo, Jinhua Yu, Wanghan Xu, Wenlong Zhang, Lei Bai, Conghui He, and Weijia Li. 2026. Earth-Agent: Unlocking the Full Landscape of Earth Observation with Agents. In *International Conference on Learning Representations*. https://arxiv.org/abs/2509.23141v3

[6] Varvara Krechetova and Denis Kochedykov. 2025. GeoBenchX: Benchmarking LLMs in Agent Solving Multistep Geospatial Tasks. In *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Generative and Agentic AI for Multi-Modality Space-Time Intelligence*. ACM, 27–35. doi:10.1145/3764915.3770721

[7] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477 doi:10.48550/arXiv.2304.11477

[8] Yanming Liu, Xinyue Peng, Jiannan Cao, Shi Bo, Yuwei Zhang, Xuhong Zhang, Sheng Cheng, Xun Wang, Jianwei Yin, and Tianyu Du. 2025. Tool-Planner: Task Planning with Clusters across Multiple Tools. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/file/7f605d59a0dbde101518b552cb616ddf-Paper-Conference.pdf

[9] Felipe Meneguzzi, Ramon Fraga Pereira, and Nir Oren. 2025. Generalised BDI Planning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. International Foundation for Autonomous Agents and Multiagent Systems, 1483–1491. doi:10.5555/3709347.3743782

[10] Qinzhuo Wu, Wei Liu, Jian Luan, and Bin Wang. 2024. ToolPlanner: A Tool Augmented LLM for Multi Granularity Instructions with Path Planning and Feedback. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 18315–18339. doi:10.18653/v1/2024.emnlp-main.1018

[11] Wanghan Xu, Xiangyu Zhao, Yuhao Zhou, Xiaoyu Yue, Ben Fei, Fenghua Ling, Wenlong Zhang, and Lei Bai. 2025. EarthSE: A Benchmark for Evaluating Earth Scientific Exploration Capability of LLMs. arXiv:2505.17139 doi:10.48550/arXiv.2505.17139

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://arxiv.org/abs/2210.03629v3

[13] Bo Yu, Cheng Yang, Dongyang Hou, Chengfu Liu, Jiayao Liu, Chi Wang, Zhiming Zhang, Haifeng Li, and Wentao Yang. 2026. GeoAgentBench: A Dynamic Execution Benchmark for Tool-Augmented Agents in Spatial Analysis. arXiv:2604.13888 doi:10.48550/arXiv.2604.13888

[14] Zian Zhai, Xingyu Tan, Gaowang Zou, Xiaoyang Wang, and Wenjie Zhang. 2026. HyperAgent: Planning and Acting over Tool-Schema Hypergraphs for Tool-Use LLM Agents. arXiv:2608.02650 doi:10.48550/arXiv.2608.02650

[15] Yongfei Zheng. 2025. Research Forefronts in Earth Science. *Chinese Science Bulletin* 70, 36 (2025), 6188–6203. In Chinese. doi:10.1360/CSB-2025-5248

A Detailed Completion Procedure

This appendix specifies the branch search and acceptance conditions supporting Section 3.4.

A.1 Directed Bridging with a Fixed Active Boundary

Suppose activated Skills A and B supply disconnected fragments. Let u be an output-boundary capability of A, with no successor among its active dependencies, and v an input-boundary capability of B, with no predecessor among its active dependencies. If inactive Skill C declares $u \rightarrow m \rightarrow v$, this branch can supply intermediate capability m without importing the entire Skill.

The active capability set is fixed at search start. Each branch must come from a single inactive Skill and contain at least one inactive intermediate capability. Membership first identifies Skills containing both endpoints; explicit dependencies or applicable conditional-ordering relations then determine directed simple paths. Co-occurrence and list order do not establish direction. An existing indirect data path from u to v excludes the candidate pair; a direct connection alone does not. Newly inserted capabilities do not recursively extend the current search.

Candidate branches must satisfy five checks:

(1) **Direction:** each adjacent pair follows an explicit dependency or applicable conditional order.
(2) **Prerequisites:** all required predecessors belong to the fixed active set or candidate branch.
(3) **Types:** upstream outputs and downstream inputs share at least one concrete type, excluding generic labels such as `Any` and `Parameter`.
(4) **Implementations:** branch operations and required predecessors have available tools within permitted scopes. The entry and external active predecessors retain their original scopes; other branch nodes use the provider Skill's constraints.
(5) **Acyclicity:** both individual and joint insertions preserve acyclic dependencies.

For each ordered endpoint pair, only a unique valid branch may be inserted automatically. Multiple candidates produce an ambiguity report and halt completion for that pair. Insertion preserves the entry instance's dependencies and tools, updates the exit's dependencies, tool scope, and bindings, and recomposes the plan. A branch is rejected if recomposition adds capabilities outside the branch or leaves mandatory dependencies unsatisfied. Algorithm 1 summarizes the procedure.

Algorithm 1 Directed Bridging from a Single Provider

Require: Plan P after compatibility completion; registered knowledge; available tools
Ensure: Updated P and acceptance/rejection records
1: Return if bridging is disabled or this plan has already been searched
2: Fix active capabilities; mark P as searched
3: Enumerate boundary pairs from different Skills; skip pairs with an indirect data path
4: Search inactive Skills for single-provider branches satisfying the five validity checks
5: Deduplicate by provider/path; reject ambiguous pairs; check joint acyclicity
6: Attempt insertion, preserving entry constraints and updating exit dependencies, tools, and bindings
7: Recompose; reject external additions or unmet mandatory dependencies

A.2 Grounding an Accepted Branch

A capability-level branch must be projected onto execution instances before it can carry data. The entry and exit must identify unique instances with an order consistent with the bridge. Required operations are inserted with their scoped implementations and all conjunctive predecessor dependencies. An endpoint ambiguity is a grounding failure, not permission to choose an arbitrary occurrence.

Each declared bridge input identifies an upstream instance and a projection from its observed output. Projection can select a named object field or a nonnegative array index; a declared list transformation can wrap an artifact as a list input. A producing step must succeed before the value becomes available. If the branch cannot be grounded consistently, the system rejects that projection rather than treating capability connectivity as executable data flow.

A.3 Candidate Skill Representation

A verified composition can be described as a candidate Skill containing its objective, applicable inputs, expected outputs, capability instances, data and control dependencies, permitted tools, and usage constraints. Task-specific paths and values become rebindable inputs where appropriate. Distinct occurrences remain distinct, and provenance links retain the conditions inherited from contributing strategies.

Registration requires checking those declarations and assigning the candidate a strategy identity before making it available for later retrieval. The proposal reuses existing capabilities and implementations; it does not create new scientific operations. Candidate packaging and registration are an extension of the workflow representation. The reported experiments do not evaluate autonomous registration, long-term accumulation, improved coverage, or reduced planning cost.

B System Architecture and Execution Details

AgentGS implements the method through an interactive frontend, an orchestration service, registered knowledge resources, and a tool execution service (Figure 3). This appendix describes how the plan in Section 3 becomes an inspectable sequence of calls.

B.1 Separation of Planning and Execution

The frontend accepts questions and resources and displays the returned answer and execution records. Orchestration retrieves strategies and tools, prepares capability requirements, invokes model planning, and validates the resulting instances. Knowledge resources provide Skill declarations, capability mappings, and tool descriptions. The execution service invokes a registered interface and returns its observed output or failure.

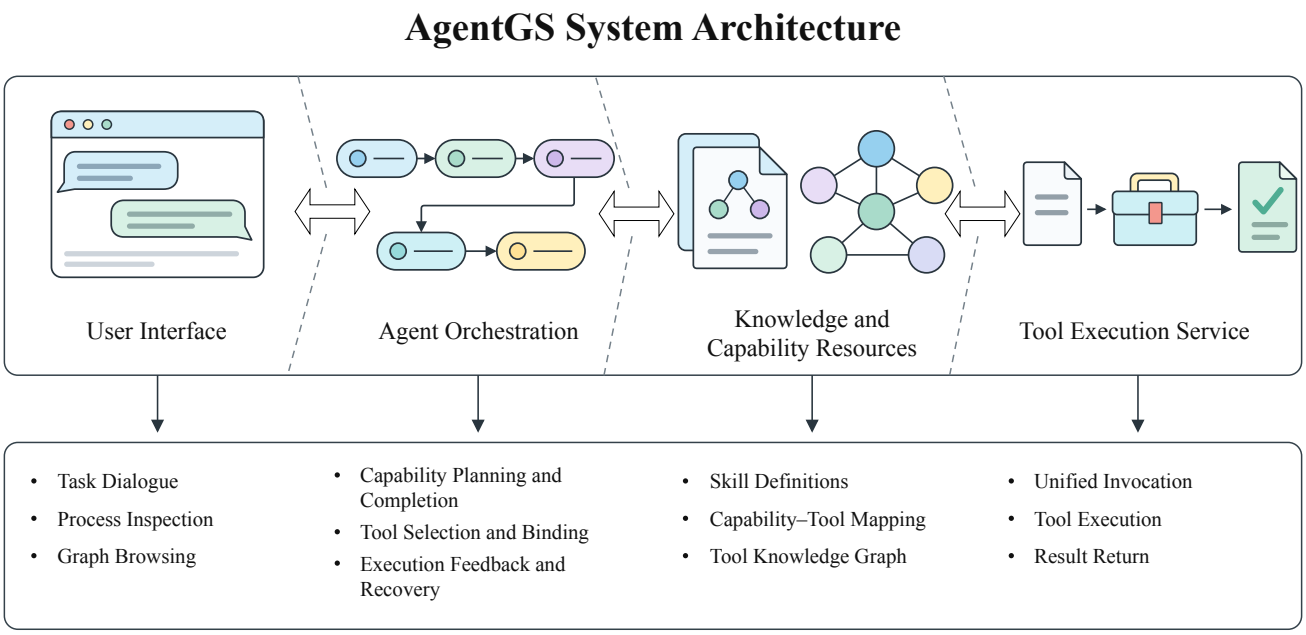


Figure 3: AgentGS architecture and main functions. The system comprises an interactive frontend, an agent orchestration service, knowledge and capability resources, and a tool execution service.

This separation allows planning to reason over declared operations while execution operates on concrete argument objects. Descriptive similarity between two tools does not by itself authorize substitution: the capability occurrence's scope and the selected interface schema still apply.

B.2 Execution Instances and Input Bindings

An execution step records its identity, tool, purpose, predecessor identities, literal arguments, and upstream input bindings. Optional Skill and Capability identifiers establish its declared scope. Unknown or forward dependencies and duplicate instance identities are rejected before execution.

A binding names a producing instance and a path into its observed structured output. The consumer receives the resolved value rather than an unverified description of what the producer was expected to return. Explicit references remain authoritative when arguments are repaired. This prevents a failed or inappropriate argument from being silently replaced with a convenient unrelated value.

Artifacts additionally retain provenance such as the producing step and tool. File and scalar bindings are validated according to their expected inputs. An operation on the wrong artifact may still execute successfully, which is why invocation success and reference-answer correctness are tracked separately.

B.3 Recovery and Reuse Boundaries

When execution recovery is enabled, the runtime can use failure feedback for argument correction and another plan attempt under configured limits. Initial model-plan repair, protocol repair, transport retries, and scientific tool retries have separate counters and scopes. Exhausting a communication budget does not establish that a scientific computation has been safely replayed.

A result can be reused across plan attempts only when the instance has unchanged semantics, fully specified inputs, and verified upstream reuse identities. The current runtime permits this for an explicit set of pure scalar computations. File-backed, implicit-input, and external-data operations are rerun rather than assumed unchanged. Matching tool names or retaining the same position in a chain is insufficient to establish reuse.

With execution recovery disabled, the first failed scientific execution ends that chain and answer generation reports incomplete work. Initial argument construction and validation remain available, so the intervention targets reaction to observed failure rather than removal of ordinary execution checks.

B.4 Knowledge Integration and Process Inspection

Integrating a tool registers its identifier, parameter schema, and input/output information, then associates it with relevant capabilities. A Skill supplies the strategy-specific dependency and implementation constraints. Runtime selection checks those declarations together with tool availability and the task's concrete inputs.

The interface exposes plans, invocation inputs and outputs, execution states, and failure feedback. Tool-graph browsing presents registered tools and associations as retrieval and inspection aids. These associations do not supersede capability dependencies or establish valid argument flow. Retained histories make it possible

to inspect whether the intended file was read, whether the correct upstream value was consumed, and which attempts contributed to the final answer.

C Dataset Sources and Construction Records

C.1 Source-Specific Task Construction

Geoscientific big-data analysis includes 40 GeoQuestions1089 questions concerning physical geographic objects and spatial operations. The remote-sensing theme retains 49 tasks from EarthBench's 248-task collection, including source questions, choices, and answers, with calls adapted to the registered interfaces. Critical-mineral and climate-change questions are selected by theme from habitable_earth_dataset; the underlying resources include MRDS, paleoclimate reconstructions, GISTEMP, and SSP warming summaries.

Earth-structure tasks use two models and an earthquake catalog. The models are LITHO1.0 and SEMUCB-WM1; earthquake records come from USGS. Energy-resource tasks use mineral, power-facility, installed-capacity, and population data. Global ocean-carbon tasks draw on SOCAT, GLODAP, and SNAPO-CO_2. Deep-time biosphere tasks use BioDeepTime, carbon-storage tasks use synthetic SCCS-OFPOSDU monitoring data, and disaster-event tasks use EM-DAT records. Synthetic carbon-storage data support reproducible operations under their supplied assumptions; they do not constitute field validation of storage safety.

C.2 Evaluation Records and Provenance

The dataset's task representation preserves one worksheet row per evaluation record. Fields identify the question and category, question text, expected answer, reference-chain information, answer type, input resource, and answer-format requirements. Stable record identities distinguish evaluation entries even when a question appears in two themes.

A source task retains its question identifier or resource locator and its reference answer. A data-derived task retains data-location information and calculations describing how the reference was obtained. Reference calls map these operational requirements to registered tools, including parameters and the relevant input/output relationships.

Construction checks record completeness and correspondence between reference-call names and registered interfaces. Sample executions check data access and invocation behavior, while computational records support recalculation. These checks are recorded separately from scored agent runs, preventing a successful construction check from being counted as an agent result.

C.3 Interface Granularity and Aggregation

The reference sequence is defined under the tool packaging available to the evaluated agents. A task wrapper may hide several operations inside one call, while another interface exposes them separately. Consequently, invocation count measures a property of the exposed workflow rather than an intrinsic scientific complexity.

Record-weighted averaging gives each of the 603 entries equal weight. A domain-balanced mean would instead give the ten themes equal weight, and a deduplicated question mean would change the treatment of repeated cross-theme questions. Those quantities answer different questions and are not interchangeable with the reported main FAS.

Numerical tolerances and answer normalization belong to the answer-type-specific scorer. Process metrics do not validate those numerical decisions. Conversely, matching a reference answer does not establish that the agent produced every requested artifact or followed the intended data dependencies.

D Experimental Provenance and Control Details

D.1 Sources of the Reported Aggregates

The main FAS table retains the answer scores supplied by the source experiment, with AgentGS identified as its first complete 603-record run. AgentGS's main TSF1/TOS are supplied by the recorded end-to-end evaluation on that population. Combining these aggregates in one table does not establish paired answer/process outcomes for every task.

The 57-record configuration study has a separate Full reference. Full's process scores are taken from the supplementary experiment, while A1–A6 use per-task records in the formal worksheet. A1's partial control, A2's inactive intervention, and A5's partial ranking change apply to those reported runs. Current implementation of a corresponding switch does not validate the identity or behavior of a past run.

GeoBenchX supplies two 202-task aggregates and uses GPT-5.6 Terra as both backbone and single reference-answer judge. The exact paired task identities and matched execution settings are not established by the aggregate table. Its 0–2 scoring scale must remain separate from GeoToolEval's answer-type-specific FAS.

The available manuscript evidence does not provide a complete set of immutable model snapshots, prompts, sampling settings, per-task budgets, source revisions, and run identifiers for all comparisons. Those values are not inferred from the current runtime defaults. The present tables report available measurements without substituting newly implemented controls for unverified historical conditions.

D.2 Interventions in the Current Runtime

Module interventions operate within Skill-guided planning. A1 preserves access to strategy declarations and admitted capabilities while requesting tool-instance planning without an ordered capability plan. This differs from a tool-only baseline that removes the Skill/Capability layer altogether.

A2 suppresses automatic mandatory-predecessor closure. A3 suppresses supporting-capability insertion and bridging, while retaining gap diagnosis. With A2 alone, completion can restore a predecessor; with A2 and A3 together, neither automatic insertion phase supplies it. The model may still explicitly include declared requirements. Effective intervention therefore needs to be evaluated through changes in preparation and planning, not assumed from a configuration flag.

A4 suppresses only directed bridging. A5 selects the first eligible implementation in a fixed tool-identifier order, preserving instance identities, constraints, and argument checks. A6 suppresses execution-driven repair and replanning after a scientific failure.

These definitions specify intended controlled comparisons; the reported A1, A2, and A5 measurements retain the limitations stated above.

Effective settings, requested interventions, and suppressed controls are recorded with execution provenance. A resume operation must preserve the same experiment identity. These records support checking whether a run applies its requested intervention but do not establish a nonzero effect on a task with no relevant dependency, alternative tool, or recoverable failure.

D.3 Candidate Access, Budgets, and Additional Baselines

A comparison of planning representations should specify whether all systems receive the same tool candidates or retain their own retrieval policies. The runtime supports exposing all registered candidates, while the retrieved setting preserves system-specific candidate selection. Routing hints and binding assistance are separate controls and can remain active even when heuristic plan generation is disabled.

A Bash-based data-only baseline changes the execution interface more substantially. A comparable data-access scope excludes materialized-query lookup tasks whose snapshot service is unavailable in that environment. A comparison that uses the full tool population for one system and a restricted accessible population for another must not interpret the difference as a planning effect. No new Bash-baseline score is asserted here.

Budgets should distinguish model requests, transport attempts, plan attempts, scientific executions, and diagnostic calls. The runtime records separate counters, but the supplied result tables do not include matched latency, token usage, or execution-cost measurements. Consequently, they establish no quantitative cost advantage.

D.4 Trace-Level Analyses Not Established by Aggregates

To relate composition to outcomes, a trace must show the selected requirements, inserted predecessors, supporting operations, and remaining gaps. To relate bridging to outcomes, it must show the provider, endpoints, accepted path, and resolved data bindings. To relate recovery to outcomes, it must distinguish failed attempts, repaired arguments, changed plans, and verified reuse.

The illustrative case provides a concrete explanation of these relationships. It is not a sampled failure distribution or a measured estimate of component utilization. Repeated matched runs and per-task alignment are needed before reporting uncertainty, significance, or causal component effects.

E Illustrative Case Study

We use an illustrative case to show how the system combines capabilities from different Skills, fills missing steps, and forms concrete tool calls. Consider the task: given several remote-sensing rasters of the same region, create and return a pixel-wise mean composite, compute the mean of its valid pixels, state whether that mean exceeds a given reference value, and report the difference. The inputs represent the same variable, their spatial grids are aligned, and the reference value uses the same units as the raster values.

Skill Retrieval returns only RasterCompose (Skill A) and ReferenceCompare (Skill B). *Skill Planning* selects InputCheck (w) and RasterMean (u) from the former, and ScalarDiff (v) and SignCheck (z) from the latter. InputCheck verifies the variable and spatial-grid requirements. RasterMean produces the mean raster from the validated image list, giving $w \rightarrow u$. ScalarDiff returns the difference between the value under comparison and the reference; SignCheck uses that difference to determine whether the value is above, equal to, or below the reference, giving $v \rightarrow z$. Hypergraph Composition expands mandatory predecessors declared by the selected strategies and retains provenance and candidate-tool scopes. At this stage, the plan consists of two disconnected fragments: compositing produces a raster file, whereas differencing requires a numerical input. The operation computing the mean of the composite is still missing.

Using u and v as shared endpoints, Missing Capability Completion retrieves $u \rightarrow m \rightarrow v$ from inactive RasterDiff (Skill C), where ValidMean (m) extracts the mean of valid pixels from the composite. Here, RasterDiff means differencing a statistic of the composite against a reference value. The query explicitly requires the mean statistic; Skill C supplies reusable dependencies and usage constraints connecting it to the preceding and following operations. Under this example's assumptions, it is the only candidate for the ordered endpoint pair satisfying prerequisite, type, tool-availability, and acyclicity requirements: u's raster output is compatible with m's input, and m's numerical output is compatible with v. Insertion preserves u's original constraints, adds m, and updates v's dependencies, candidate-tool scope, and input bindings. Existing dependencies $w \rightarrow u$ and $v \rightarrow z$ remain. Recomposition and dependency verification connect the fragments into input validation, mean compositing, valid-pixel mean statistics, scalar differencing, and sign comparison: $w \rightarrow u \rightarrow m \rightarrow v \rightarrow z$.

Tool Grounding then selects validation, compositing, statistics, differencing, and sign-comparison tools within each instance's candidate scope and parameter constraints. GeoToolKG associations guide candidate retrieval, but concrete invocations still require instance and input-compatibility checks. Grounding a bridge into calls additionally requires that u and v each correspond to exactly one endpoint instance and that their order agrees with the bridge direction. During execution, the validated image list is passed to the compositing tool. The statistics tool receives a list containing only the composite raster's path. The returned mean and the task's reference value are bound to differencing parameters a and b, producing $d = a - b$. Sign comparison then compares d with zero. The statistical object remains the composite created for this task, the comparison baseline remains the supplied reference, and the final relation is based on the same difference calculation.

If the statistics call fails, recovery can repair the invocation using the observed failure and the applicable scope. A local retry can continue to refer to the composite produced earlier in the current plan attempt. A new plan attempt does not automatically reuse file-backed computations: their state must be re-established according to the execution policy. This distinction prevents the illustration from implying an unrestricted cache of prior artifacts.

Successful execution yields the composite raster, its valid-pixel mean, the numerical difference, and the comparison relation. These outputs must refer to the same composite and reference value.

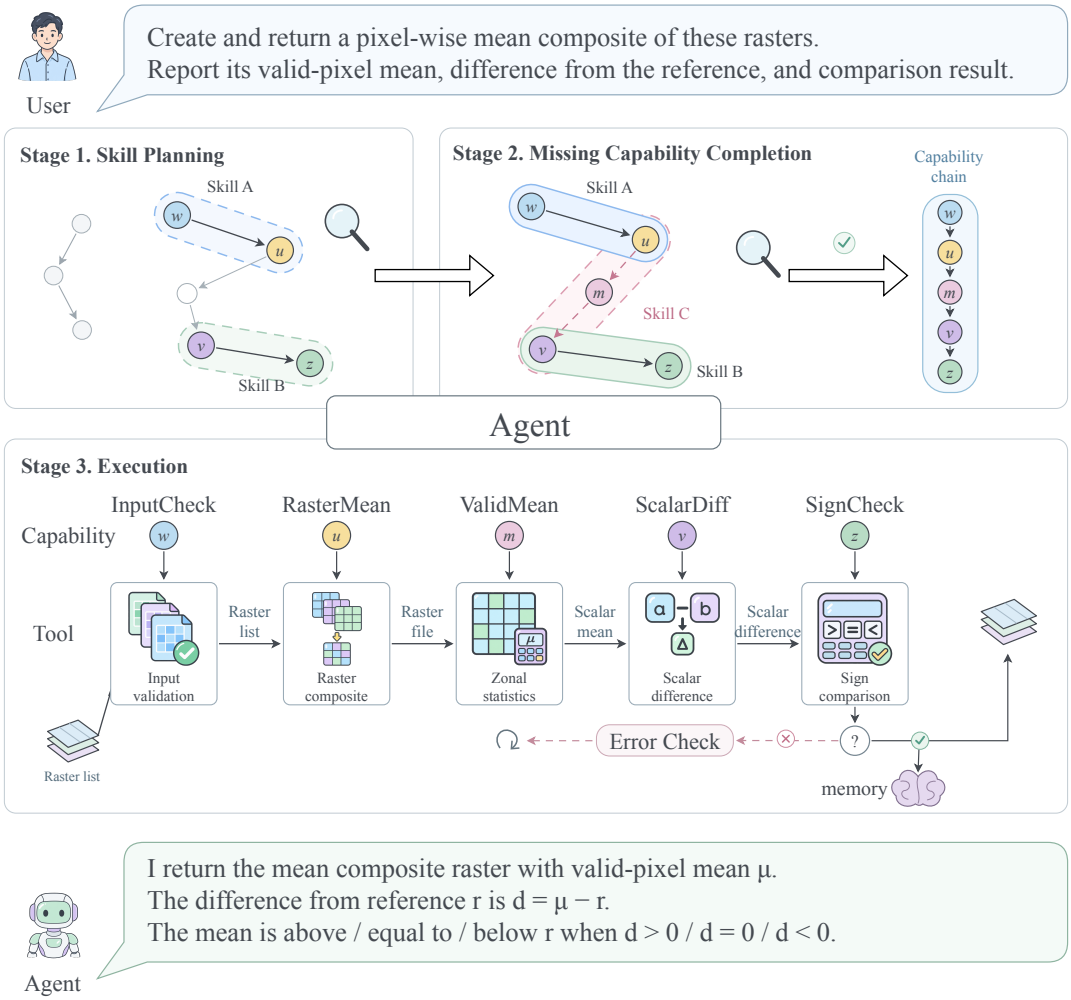


Figure 4: Cross-Skill capability composition and tool execution. Stage 1 selects $w \rightarrow u$ from Skill A and $v \rightarrow z$ from Skill B. Stage 2 inserts valid-pixel mean statistics m using Skill C's $u \rightarrow m \rightarrow v$ branch. Stage 3 grounds the five-step workflow in tool calls, propagates intermediate artifacts, and returns the composite raster, mean, difference, and comparison relation. The workflow is a mechanism illustration.

Merely selecting the five expected tool names would not establish that consistency; the artifact bindings are the operational link.

After verification, the composition can be described as a candidate CompositeCompare Skill (D), parameterizing the image list and reference value while retaining dependencies and usage constraints. The example illustrates how a new strategy can be represented over existing capabilities. It does not provide an empirical result on registration, reuse across tasks, or improved planning efficiency.

F Limitations and Further Evaluation

F.1 Dependence on Registered Knowledge

Composition depends on the coverage and quality of Skill declarations, dependency relations, and implementation mappings. A hypergraph can connect known operations but cannot supply a scientific operation absent from the registry. Incomplete or inaccurate constraints can prevent useful reuse or admit an unsuitable composition. Dedicated knowledge construction for GeoBenchX is part of the evaluated deployment and should be included when considering the cost of adopting another tool environment.

F.2 Structural Validity and Scientific Meaning

Type compatibility and dependency closure establish declared workflow conditions. They do not prove that a selected statistic, spatial resolution, physical model, or comparison baseline is scientifically appropriate. Scientific validation also depends on data quality, units, assumptions, and the meaning of the question. The framework makes bindings and dependencies inspectable, but expert or domain-specific checks can still be necessary.

F.3 Benchmark Scope and Metric Sensitivity

GeoToolEval has unequal theme sizes, repeated cross-theme questions, and interface-dependent chain lengths. Its record-weighted mean therefore reflects the chosen population and wrappers. Remote sensing dominates the longest-chain group, and some single-call tools encapsulate several operations. These factors limit generalization from chain length to intrinsic task difficulty.

Name-based process metrics favor agreement with the reference implementation. They may penalize a valid alternative tool or order and may miss incorrect parameters in an otherwise matching chain. Final-answer scores address a different aspect of quality and can also miss missing artifacts when the reported scalar or choice happens to agree. Complete assessment requires evaluating the requested outputs and their provenance.

F.4 Experimental Attribution and Reuse

The reported configuration studies contain incomplete interventions and heterogeneous result provenance. Aggregate differences cannot isolate all proposed module contributions, and no repeated-run uncertainty or matched efficiency analysis is supplied. External evaluation uses one model judge and requires verification of paired task identity and settings.

Candidate Skill representation provides a way to retain a verified composition. The present experiments do not demonstrate autonomous Skill registration, improved future-task coverage, or reduced planning cost. Evaluating reuse requires separating construction tasks from later tasks and checking that applicability and scientific assumptions remain valid under new inputs.